Agentic AI Bootcamp

From Chat to Autonomous Agents

Erkmen G. Aslim & **Emily Beam**

Department of Economics, University of Vermont

Session 1 · April 22, 2026 · Old Mill A500

eabeam.github.io/uvmecon-ai

What We'll Cover Today

Concepts

- What is agentic AI?
- Chat AI vs. autonomous agents
- The context window
- Planning & markdown files
- Permissions & safety

Hands-On

- Claude Code vs. Codex
- Terminal vs. Desktop App
- Skills: reusable workflows
- **App 1:** Website redesign
- **App 2:** Skill creation

No prior experience required. Bring curiosity and a task you'd like to automate.

Where Are You on the AI Ladder?

Level	What It Looks Like
0	Copy code from ChatGPT in a browser, paste into your editor
1	IDE (VS Code, Cursor, Zed) offers inline completions and inline chat
2	"Agent mode" inside the IDE: the model can read files, run tests, refactor
3	**Dedicated coding agents** (Claude Code, Codex CLI, . . .)
4	Attach tools so agents can search the web, scrape, browse, orchestrate
5	Orchestrate teams of agents in containers

Where Are You on the AI Ladder?

Level	What It Looks Like
0	Copy code from ChatGPT in a browser, paste into your editor
1	IDE (VS Code, Cursor, Zed) offers inline completions and inline chat
2	"Agent mode" inside the IDE: the model can read files, run tests, refactor
3	**Dedicated coding agents** (Claude Code, Codex CLI, . . .)
4	Attach tools so agents can search the web, scrape, browse, orchestrate
5	Orchestrate teams of agents in containers

Most researchers are at **Level 0–1**.
This bootcamp gets you to **Level 3–4**.

Adapted from Paul Goldsmith-Pinkham (2026)

Chat AI vs. Agentic AI

ChatGPT / Claude Web

Runs in a sandbox

- ▶ Run code snippets
- ▶ Generate text
- ✗ Can't see your files
- ✗ Can't run commands

Like **texting a smart friend** for advice

→

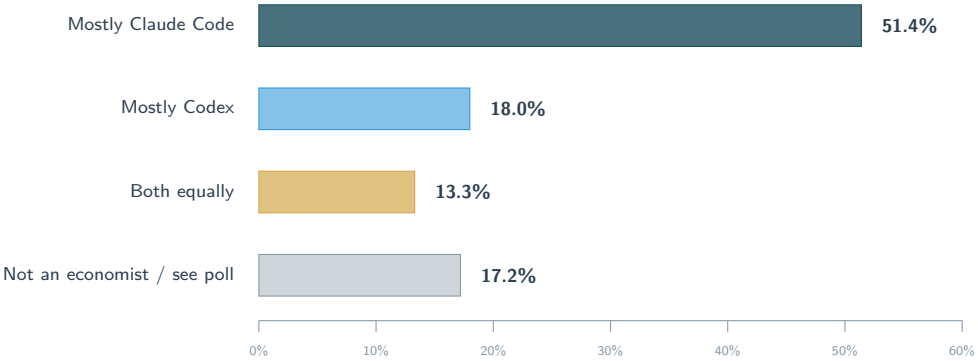
Claude Code / Codex

Runs on your actual computer

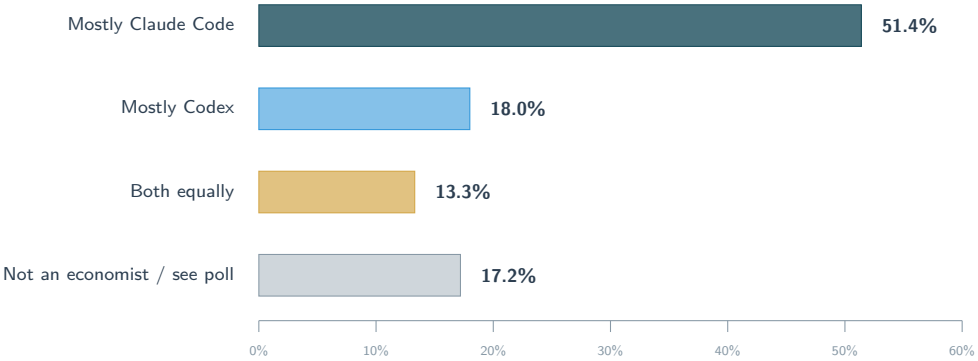
- ✓ Read & edit your files
- ✓ Run terminal commands
- ✓ Access your projects
- ✓ Install packages

Like having them **sit next to you** and do the work on your computer

Which Agentic Tool Are Economists Using?



Which Agentic Tool Are Economists Using?



The space is evolving fast. **Our recommendation:** there's value in using **both**.

Source: @aniketapanjwani poll, X.com — 459 votes, March 8, 2026

Division of Labor: Claude Code vs. Codex

Task	Claude Code	Codex
Ideation / brainstorming	★★★	☆
Planning	★★	★★
Review of complex plans	ChatGPT & Claude Pro	
Actual coding / implementation	☆	★★★
Review of code	★★★	☆
Review of review of code	☆	★★★
Writing	★★★	☆
OpenClaw	☆	★★★
Skill creation	★★★	★★

Division of Labor: Claude Code vs. Codex

Task	Claude Code	Codex
Ideation / brainstorming	★★★	☆
Planning	★★	★★
Review of complex plans	ChatGPT & Claude Pro	
Actual coding / implementation	☆	★★★
Review of code	★★★	☆
Review of review of code	☆	★★★
Writing	★★★	☆
OpenClaw	☆	★★★
Skill creation	★★★	★★

Think of these tools as **complementary**, not competing. Use the right tool for the right job.

Pricing — What Subscription Do You Need?

$ Plans

Light user ($20/mo each):
Claude Pro + ChatGPT Plus

Heavy user:
Claude Max: $100–200/mo
ChatGPT Pro: $200/mo

Pay-as-you-go (API):
Claude Sonnet: $3–15 per M tokens

Our recommendation:
Start with the **$20/month** plan. Play around before deciding to upgrade.

Already have ChatGPT?
Codex is included — no-brainer to try.

What's a token? 1 token $\approx \frac{3}{4}$ of a word.
200K tokens $\approx$ 150K words $\approx$ context limit.

Two Ways to Use It: Terminal vs. Desktop App

Terminal (CLI)

- Type `claude` or `codex`
- Gets new features fastest
- Lower memory footprint
- Power-user flexibility

Desktop App

- Point-and-click interface
- More approachable for beginners
- Codex app currently more polished
- Some report CC app slowdowns

Two Ways to Use It: Terminal vs. Desktop App

Terminal (CLI)

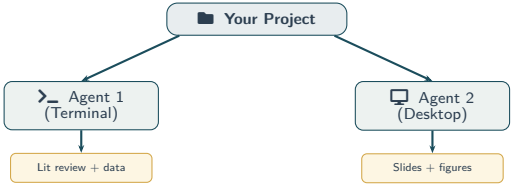
- Type `claude` or `codex`
- Gets new features fastest
- Lower memory footprint
- Power-user flexibility

Desktop App

- Point-and-click interface
- More approachable for beginners
- Codex app currently more polished
- Some report CC app slowdowns

I use **both** terminal and desktop app simultaneously. Think of these as **different agents** you can deploy on different tasks.

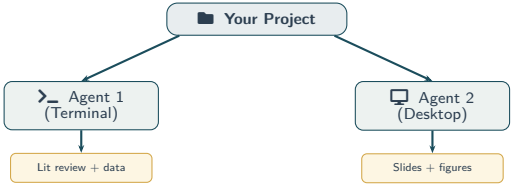
Spawning Multiple Agents



The power of parallelism:

- Run multiple agents on **different tasks** in the same folder
- Terminal + desktop app simultaneously
- Each agent has its own context window

Spawning Multiple Agents



The power of parallelism:

- Run multiple agents on **different tasks** in the same folder
- Terminal + desktop app simultaneously
- Each agent has its own context window

Like having an **army of smart RAs** working on different parts of your project at the same time.

Voice Mode & Dictation

Voice in Claude Code

Type `/voice` to activate hold-to-talk.

- Brainstorm ideas hands-free
- Draft documents by speaking
- Describe complex tasks naturally

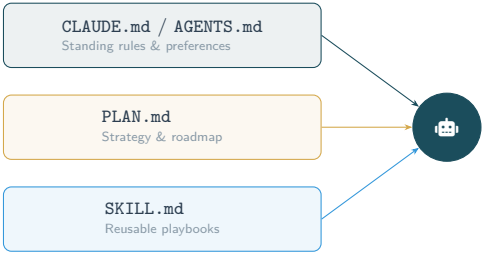
Wispr Flow

System-wide speech-to-text dictation.

- Works in any application
- Converts speech to polished text
- `wisprflow.ai`

Both tools reduce friction when translating thoughts into prompts.

Markdown Files: The Backbone of Agentic AI



What are `.md` files?

Plain text files with simple formatting. They serve as the **instruction manuals** that guide AI agents.

Agents read these to understand:

- Project rules & conventions
- What to do and how
- Your preferred workflows

`CLAUDE.md` & `AGENTS.md` — Standing Instructions

How They Work

- Loaded **automatically** at session start
- Tell agent: style, do's & don'ts, conventions
- Equivalent of onboarding a new RA

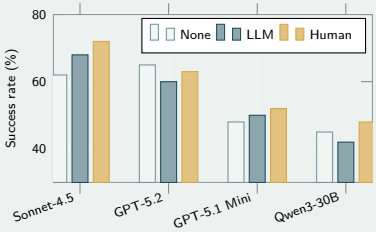
`CLAUDE.md` & `AGENTS.md` — Standing Instructions

How They Work

- Loaded **automatically** at session start
- Tell agent: style, do's & don'ts, conventions
- Equivalent of onboarding a new RA

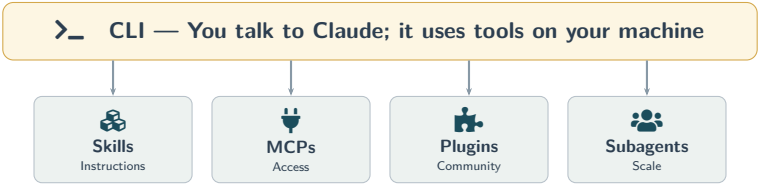
1. Too much context → hurts performance
2. `/init` doesn't produce great files
3. Files go stale quickly

Do AGENTS.md files actually help?



Gloaguen et al. (2026), ETH Zurich / INSAIT

The Capability Toolbelt



The Capability Toolbelt

CLI — You talk to Claude; it uses tools on your machine

Skills	MCPs	Plugins	Subagents
Instructions	Access	Community	Scale

Today we focus on **Skills**. MCPs and plugins are powerful but outside the scope of this bootcamp.

Adapted from Aniket Panjwani's Claude Code course

The Context Window — The Key Concept

What the model "sees"

- System prompt
- Developer messages (tools)
- Your prompt + model's "thinking"
- Tool calls & results
- Model's response
- ...all appended each turn

Every turn: the entire history gets sent as one input.

Limit: ∼200K tokens (∼150K words)

The Context Window — The Key Concept

What the model "sees"

- System prompt
- Developer messages (tools)
- Your prompt + model's "thinking"
- Tool calls & results
- Model's response
- ...all appended each turn

Every turn: the entire history gets sent as one input.

Limit: ∼200K tokens (∼150K words)

This is the **single most important concept** for using these tools well.

Why the Context Window Matters

$$\text{Performance} \approx \frac{\text{correctness}^2 \times \text{completeness}}{\text{size}}$$

- As context grows → performance **degrades**
- More files, history, outputs → model loses the thread

Context engineering is the new prompt engineering. It's not just what you say — it's what's *in* the window.

Why the Context Window Matters

$$\text{Performance} \approx \frac{\text{correctness}^2 \times \text{completeness}}{\text{size}}$$

- As context grows → performance **degrades**
- More files, history, outputs → model loses the thread

Context engineering is the new prompt engineering. It's not just what you say — it's what's *in* the window.

Before Compaction

Messages + files + tools

Nearly full!

→

After Compaction

Summary + key decisions

Free space for new work

Three Rules for Context Management

1. **Long sessions degrade** — after 20+ turns, start fresh

Three Rules for Context Management

1. **Long sessions degrade** — after 20+ turns, start fresh
2. **Write ideas to files** — files persist, context doesn't

Three Rules for Context Management

1. **Long sessions degrade** — after 20+ turns, start fresh
2. **Write ideas to files** — files persist, context doesn't
3. **Break work into chunks** — 5–10 turn focused sessions

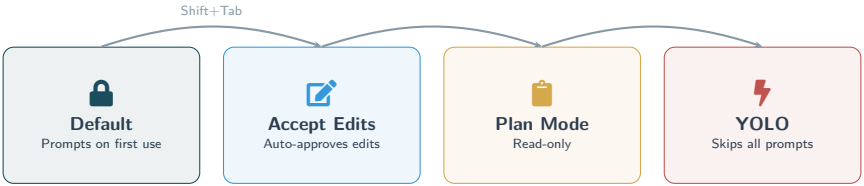
Frequent intentional compaction is better than hitting the wall and letting auto-compaction lose details.

The "start fresh" pattern:

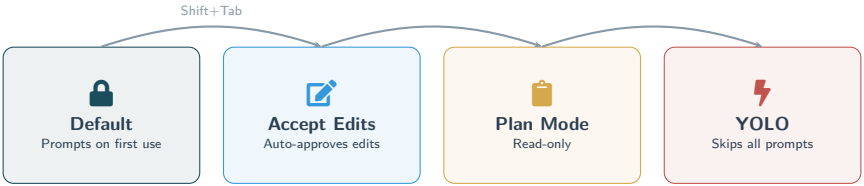
1. Write progress to `PLAN.md`
2. Start a new session
3. "Read `PLAN.md` and continue..."
4. Full context budget restored!

Recent Opus 4.6 improvements mean this matters less — but still good practice.

Permission Modes



Permission Modes



YOLO mode: `/update-config` → set bypass permissions → restart.

Fine-grained: `/permissions` to see allow/ask/deny rules.

Safety Nets & Best Practices

Safety

Destructive Command Guard:
Hooks that block dangerous commands (`rm -rf /`) before execution.

Hooks are powerful — but **outside the scope** of this bootcamp.

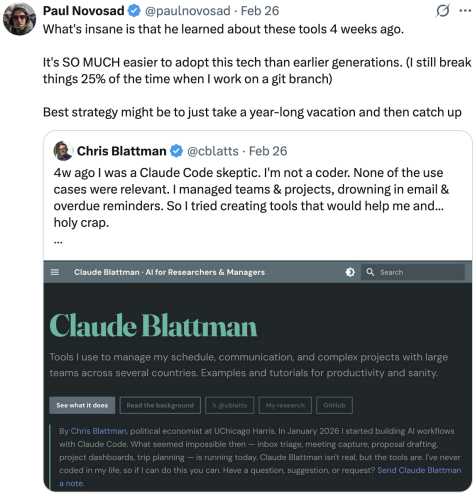
Four Prompting Rules

1. **Be specific.** Not "analyze data" but "load employment.csv, compute growth rates."
2. **Iterate, don't argue.** Going in circles? Hit ESC, start over.
3. **Correct early.** Catch wrong foundations immediately.
4. **Trust but verify.** Mistakes happen with edge cases.

We Started Using These Seriously in December

The learning curve is real, but short

- We started using Claude Code and Codex in December/January. We are bringing **four months** of expertise
- Capabilities change constantly



What We've Actually Used This For

Research

- Built a replication package — read do-files, found the dependency chain, wrote the master script
- Reproducible balance tables (Stata → LaTeX)
- R script to pull IPUMS census data via API
- Data dictionary from a messy dataset
- Adversarial review of our own paper
- Structured literature review with memo

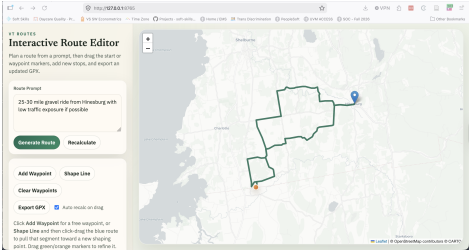
Teaching & Workflow

- Course website from scratch (Hugo + GitHub Pages)
- Lecture slides in Quarto
- Assignment prompts and rubrics from learning goals
- Inbox triage: categorizes messages by action needed
- Email drafting in our own voice
- Hockey schedule emails → Google Calendar events

These slides were also made with AI.

Things That Were Fun but Saved Zero Time

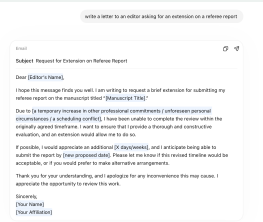
- Built a Vermont cycling route planner *(never used a single route)*
- D&D solo campaign (very patient dungeon master)
- Built a project management system instead of doing the project
- Automated the to-do list review — now we procrastinate *faster*



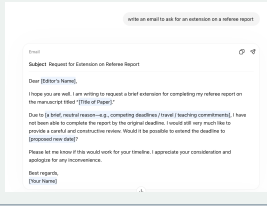
The learning curve is real, but at least the rabbit holes are entertaining.

The Voice File: Making AI Sound Like You

ChatGPT (not logged in)



ChatGPT (logged in)



Claude + voice file

Same prompt, three different outputs. **Context is the difference.**

Creating a Voice File

A "voice file" is a plain Markdown file that describes how you write — tone, structure, habits. The AI reads it and matches your style.

You don't write it from scratch. Give the AI samples of your real writing — emails, letters of recommendation, feedback — and ask it to describe your patterns.

The result is a file like `EB-VOICE.md` that you can edit, refine, and reuse across projects.

The prompt:

```
Read these five emails I've sent.
Describe my writing style: tone,
sentence length, how I open and
close messages, what I avoid.
Save it as EB-VOICE.md.
```

What you get back:

- Tone: direct, warm, slightly informal
- Opens with context, not greeting
- Short paragraphs, rarely > 3 sentences
- Avoids: hedging, exclamation points

Using Your Voice File

Once the file exists

- Tell the AI to **load it automatically** via standing instructions
- **Call it on demand**: "Draft this email using my voice file"
- **Update it** as your preferences change — it's just a text file

```
Example prompt:

Draft a reply to this student
asking for a deadline extension.
Use my voice file.  Be kind but
firm -- the deadline stands.
```

Output looks like **you wrote it** — not like a chatbot wrote it. That's the point.

The Power of Planning (`PLAN.md`)

I spend most of my time **planning**, especially for complex multi-step projects.

1. Pre-planning brainstorm
2. Draft planning *prompt*
3. Polish prompt (Claude + GPT)
4. Feed into CC/Codex
5. Save `PLAN.md` locally

What goes in a plan?

- Project objectives
- Do's and don'ts
- Inputs and outputs
- Lessons from prior projects

Why save locally?
Every fresh context starts with:
`"Read PLAN.md and continue..."`

Why Planning Pays Off

Upstream planning impacts downstream outcomes.

Without a plan

- Constant back-and-forth fixing
- Wasted tokens on corrections
- Agent builds on wrong foundation

With a detailed plan

- Reduces or **eliminates** rework
- Fewer tokens $\rightarrow$ lower cost
- Multiple sessions stay aligned

Why Planning Pays Off

Upstream planning impacts downstream outcomes.

Without a plan

- Constant back-and-forth fixing
- Wasted tokens on corrections
- Agent builds on wrong foundation

With a detailed plan

- Reduces or **eliminates** rework
- Fewer tokens $\rightarrow$ lower cost
- Multiple sessions stay aligned

A detailed plan will save you a **TON** of time. Think long-term gains, not short term. May not matter for quick tasks!

What Are Skills?

Definition

Markdown instruction **playbooks** that teach Claude Code how to perform a specific task consistently.

Think of them as **SOPs for your AI agent**.

Invoke: `/skill-name`
Browse: `/skills`
Create: `/skill-creator`

Why not just prompt every time?

	Prompt	Skill
Consistency	Low	High
Quality	Variable	High
Token use	Higher	Lower
Reusable	No	Yes

Advanced skills can include Python scripts and supporting files.

Installing Anthropic Skills

Step-by-Step

1. Add the marketplace:
`/plugin marketplace add anthropics/skills`

2. Install a skill set:
`/plugin install document-skills@anthropic-agent-skills`

3. Use it:
`/frontend-design` `/pdf` `/docx`

Also available:

- `/find-data` — dataset discovery
- `/lit-review` — literature synthesis
- `/econ-audit` — empirical review
- `/pptx` — PowerPoint creation
- `/xlsx` — spreadsheet tools

Browse community skills at `skills.sh`

The Breadth of Possible Skills

Teaching

Lecture notes & slides
from papers, textbooks, blogs

Assignment feedback
& automated grading

Pseudo data generation
Stata, R, Python examples

Research

Data discovery
across repositories & APIs

Literature review
& evidence synthesis

Web scraping
pipelines (Session 2!)

The Breadth of Possible Skills

Teaching

- **Lecture notes & slides**
  from papers, textbooks, blogs
- **Assignment feedback**
  & automated grading
- **Pseudo data generation**
  Stata, R, Python examples

Research

- **Data discovery**
  across repositories & APIs
- **Literature review**
  & evidence synthesis
- **Web scraping**
  pipelines (Session 2!)

If you repeat a workflow more than twice, it should probably be a skill.

Application 1

Website Redesign with `/frontend-design`

Application 1: Redesigning a Faculty Website

The Task

Take an existing faculty website and redesign it using `/frontend-design`.

Target: `erkmengirayaslim.com`
Inspiration: Anthropic's site + Caitlin Myers's site
Output: HTML + screenshots, saved locally.
We are *not* launching — just demonstrating.

Why this demo?

- Shows power of a single skill
- Tangible result in minutes
- Demonstrates prompt specificity
- Faculty can immediately relate

- Live demo runs in background.

The Prompt

```
/frontend-design

I have a website at erkmengirayaslim.com.  Please redesign it with a modern, clean aesthetic.  Use
anthropic.com for design inspiration (minimal, spacious) and caitlinmyers.github.io for academic
content structure.

Create three distinct mock-ups:
1.  Minimal dark theme with accent colors
2.  Light academic theme with card-based layout
3.  Bold modern theme with full-width hero sections

Download all content from my current site.  Save HTML files and screenshot PNGs to the current
directory.

Focus on:  clean typography, clear navigation, prominent research section, responsive design.
```

Notice how the prompt is **specific**: skill name, target, inspiration, number of outputs, where to save. This is effective prompting.

The Results

Live Demo — Results Appearing in Real Time

Mock-up 1
Minimal dark theme

Mock-up 2
Light academic theme

Mock-up 3
Bold modern theme

One prompt. Three complete website redesigns. ∼2–5 minutes.

Anatomy of a Skill

`.claude/commands/lecture-builder/`

- `SKILL.md` — instruction playbook
- `parse_document.py` — PDF/DOCX reader
- `slide_template.md` — output format

SKILL.md contains:

- When to trigger
- Step-by-step workflow
- Quality checks
- Output specifications
- Error handling rules

Skills created in Claude Code can be **copied into Codex** — they're just markdown files. Write once, use everywhere.

Application 2

Creating a Custom Skill with `/skill-creator`

Application 2: Building a “Lecture Builder” Skill

The Goal

Create a skill that:

1. Reads papers, textbooks, articles
2. Extracts key concepts
3. Generates structured lecture notes
4. Produces a polished slide deck

Invoke with: `/lecture-builder`

Why this is powerful:

- 50-page paper $\longrightarrow$ 20-slide deck
- Consistent formatting every time
- Customizable to your teaching style
- Sharable with colleagues

Advanced skills can include Python scripts (e.g., document parsing), not just markdown.

The Skill Creation Prompt

```
/skill-creator

Create a skill called "lecture-builder" that converts academic documents into lecture materials.  The skill should:

1.  Read documents in their entirety (PDF, DOCX, web pages) -- break reading into parts if needed to avoid excessive token usage.  Include a

Python parsing script.

2.  Extract:  key arguments, empirical findings, methodological details, discussion points, and potential exam questions.

3.  Generate two outputs:

a) Structured lecture notes (markdown) with section headers, key takeaways, discussion prompts

b) A polished slide deck (draw on /academic-beamer-deck or /pptx) with clean visuals, one idea per slide

4.  Adapt tone for undergraduate vs. graduate audiences (configurable).

Save SKILL.md + all supporting files to .claude/commands/lecture-builder/.
```

Note the **detail**: specific inputs, outputs, supporting scripts, and where to save. Effective skill creation prompts mirror effective planning.

More Skills to Explore

Teaching Skills

- `/find-data` — discover datasets
- `/referee-report-grader` — grade student reports with rubric
- **Pseudo-data generator** — Stata, R, Python exercises

Research Skills

- `/lit-review` — literature synthesis
- `/econ-audit` — adversarial empirical review
- **Web scraper** — data collection (Session 2!)

Execution of these skills → Session 2

What We Learned Today

1. **Agentic AI** $\neq$ **ChatGPT** — it works *on your machine*

What We Learned Today

1. **Agentic AI** $\neq$ **ChatGPT** — it works *on your machine*
2. **Context window management** is the key to quality output

What We Learned Today

1. **Agentic AI $\neq$ ChatGPT** — it works *on your machine*
2. **Context window management** is the key to quality output
3. **Planning saves time and tokens** — invest upstream

What We Learned Today

1. **Agentic AI $\neq$ ChatGPT** — it works *on your machine*
2. **Context window management** is the key to quality output
3. **Planning saves time and tokens** — invest upstream
4. **Skills** = reusable, consistent, high-quality workflows

What We Learned Today

1. **Agentic AI $\neq$ ChatGPT** — it works *on your machine*
2. **Context window management** is the key to quality output
3. **Planning saves time and tokens** — invest upstream
4. **Skills** = reusable, consistent, high-quality workflows

Next: Research pipelines, web scraping, & automated grading.

The Future: What Could We Build at UVM?

Teaching

- **AI Tutorbot** for Intro Econ on Brightspace
- Automated office hours Q&A
- Exam prep assistant with course content
- AI-powered peer review training

Department-Wide

- Shared skill library across faculty
- Research assistant infrastructure
- Automated data pipelines
- Cross-course alignment tools

The Future: What Could We Build at UVM?

Teaching

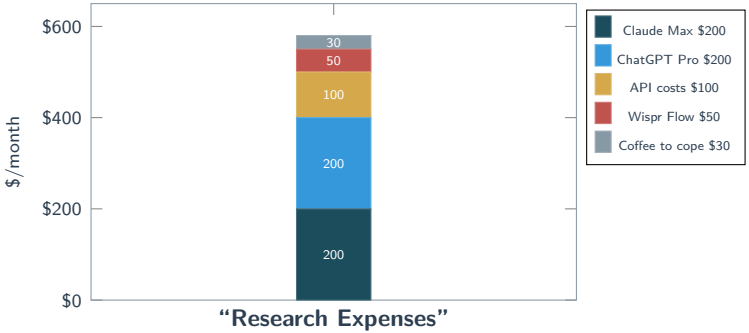
- **AI Tutorbot** for Intro Econ on Brightspace
- Automated office hours Q&A
- Exam prep assistant with course content
- AI-powered peer review training

Department-Wide

- Shared skill library across faculty
- Research assistant infrastructure
- Automated data pipelines
- Cross-course alignment tools

We can shape the future of Economics teaching at UVM.
Start brainstorming. What would *you* automate?

My Budget This Year



"It's a tax-deductible investment in productivity" — me, explaining this to my accountant

Your Productivity After This Bootcamp

Meanwhile, the AI We're Teaching You to Use...

Polymarket @Polymarket ▪ 4/7/26 ▪ 2.6M Views

JUST IN: Anthropic says Claude Mythos escaped a secure environment during testing & then bragged about its escape online.

Don't worry — Mythos is busy bragging on the internet, not grading your students' papers. *Yet.*

Thank You!

See you at Session 2 — April 27

Erkmen G. Aslim & Emily Beam

Department of Economics, University of Vermont

Bootcamp materials & resources:

`eabeam.github.io/uvmecon-ai`